

The Agentic Task Ecosystem: A Supply-Side Record of Agentic Automation

Zanele Munyikwa¹, Campbell Lund¹, Thomas Euyang¹, Aidan Peppin²,
and Marzieh Fadaee¹

¹Cohere Labs, ²Cohere

Corresponding authors: {[zanele.munyikwa](mailto:zanele.munyikwa@cohere.com), [campbell.lund](mailto:campbell.lund@cohere.com), [marzieh](mailto:marzieh@cohere.com)}@cohere.com

Summary

Large language models increasingly use external tools to act on digital systems, but there is little systematic evidence about which kinds of work developers are making available to them. We present the Agentic Task Ecosystem (ATE), a tagged, open dataset of 694,411 deduplicated tools from 123,069 public MCP server listings. As a supply-side measure, ATE is an early signal, both preceding and complementary to evidence on AI exposure and usage data.

To build ATE, we combine top-down matching to O*NET tasks with a bottom-up taxonomy of the full corpus to study both where agent tooling is emerging and how developers are defining work for machines. Under our strict matching standard, only 2.6% of tools map cleanly onto a complete O*NET task. The bottom-up taxonomy shows what this match rate leaves out. Some categories represent task components, while others span or bundle several recorded tasks. A small set describes work with no O*NET precedent, much of it involving the management of agents. Throughout our crosswalk, we find a measurement error between the units for describing human-work (e.g. task statements), and machine-work (e.g. tool descriptions), the latter existing at a much finer grain. This has implications for task-level exposure measures and bounds automation activity rather than counts it.

Across 178 occupations, we find that matched tasks fall near the midpoint of an occupation's expertise distribution on average, and the amount of tool coverage does not predict where they fall. However, this average conceals differences across occupational groups. Tools reach relatively specialized tasks in computing and healthcare occupations and relatively routine tasks in production and legal occupations. Further analysis suggests that tools reach further into specialized work when working with digital information is central to an occupation's expertise.

ATE is bounded by O*NET's incomplete representation of recent and fine-grained work and by our focus on public tools. Future research should examine expertise beyond direct matches, how tools chain into longer workflows, and how these patterns extend into bespoke internal workflows.

1 Introduction

Existing measures seeking to quantify the impacts of AI on work generally take one of two vantage points. First, exposure studies estimate which tasks AI could perform and aggregate those judgments back to occupations [Eloundou et al. \(2023\)](#); [Felten et al. \(2021\)](#). The second vantage is usage. Studies classify the kinds of tasks that people bring to AI systems and the degree of automation observed in those interactions [Handa et al. \(2025\)](#); [Tomlinson et al. \(2025\)](#); [Chatterji et al. \(2025\)](#); [Isenko et al. \(2026\)](#). Taken together, evidence on exposure combined with downstream usage make up the bulk of our understanding of how AI is changing, and will continue to change the nature of work. However, theoretical measures of capability do not tell us what parts of work are automatable in practice ([Lund et al., 2026](#)), and usage studies do not reflect how these capabilities actually enter the workplace through products, tools and workflows. Between modeled capability and observed demand lies a third object of study which isolates the kinds of work developers have already packaged for AI systems to take autonomous actions. We call this third, complementary piece of evidence, the **Agentic Task Ecosystem (ATE)**.

Technically speaking, ATE is a measure of public Model Context Protocol (MCP) directories, crosswalked to work tasks. Today’s large language models, which users often interface through chat applications, rely on external tools to complete tasks. These tools allow models to query databases, modify files, execute code, interact with enterprise software, and coordinate multi-step workflows. In sum, external tools extend a model’s outputs beyond generated information to actions in external systems. The Model Context Protocol is an open standard through which developers expose tools and data sources to language models, and since its release, public directories have accumulated a large and rapidly changing set of machine-callable functions.

Connecting public agent tools to occupational tasks adds a supply-side measure to the study of automation and augmentation. Task-based accounts distinguish between tasks transferred to machines and new tasks that increase demand for human labor [Autor et al. \(2022\)](#); [Acemoglu & Restrepo \(2019\)](#). Yet empirical measures generally begin from taxonomies of *human work* and ask whether AI can perform the tasks they record. In this study, we do not start with the assumption that machine work will mirror human work. Rather, we start by understanding the actions made available to machines, and then ask how these functions relate to the existing structure of human work.

Each published tool in ATE is a dated record of functionality that a developer chose to make available to an AI system. These tools expose how developers decompose work for machines, and while a tool defines a particular unit of action, that unit does not necessarily correspond to the same task boundaries we use to describe human work. A tool may represent an entire occupational task, one small component of it, or an action spanning several recorded tasks. Our crosswalk of tools to tasks exposes a measurement problem, as we find that the vast majority of automatable actions do not inherit human task boundaries. Measuring AI automation against existing occupational task statements not only systematically misses the creation of new, undocumented work, but we also find that it misses matches due to granularity and the composition of agentic work itself.

The ATE dataset is the most expansive, publically available view into the tools and tasks for autonomous AI action. To build it, we aggregate 123,069 distinct MCP server listings from seven public directories observed in May 2026. Those listings contain 696,291 published tool listings, yielding 694,411 tools after deduplication, which we crosswalk to O*NET work tasks. To be precise, ATE measures published supply. It does not establish whether a tool has been deployed, adopted in a workplace, used successfully in a long-horizon workflow, or shown to be reliable. The presence

of a tool in ATE indicates that a builder considered a function reliable enough to make available to an AI system, which allows us to monitor early signals *before* downstream evidence about adoption or workplace use.

In this paper, we use ATE to ask four questions: (1) How much published agent-tool supply corresponds directly to recorded occupational tasks? (2) What forms of new work appear when categories are derived bottom-up rather than imposed from occupational taxonomies? (3) What predicts where tool supply emerges? And, (4) when tools map to existing work, where do they fall within an occupation’s distribution of expertise?

We summarize our findings:

1. **Direct coverage:** Under a standard that requires a tool to perform a recorded task from end to end, 18,058 of 694,411 tools (2.6%) are judged to match. The top ten occupations account for 44% of matched tools, and 419 of 923 O*NET occupations show no activity under either our direct or taxonomy-based measure.
2. **Beyond direct matches:** Clustering tool descriptions reveals finer-grained forms of recognized work, infrastructure used to operate agents, and a small set of categories with no identified O*NET precedent. The low direct-match rate reflects a mismatch between the units in which human work is recorded and those in which functionality is packaged for machines.
3. **Correlates of tool supply:** Theoretical exposure and realized coverage are correlated at 0.54, but exposure does not tell us which kinds of tasks have tool matches. We find no significant correlation between exposure and the position of matched tasks within an occupation’s expertise distribution. In the 65-occupation WORKBank sample (Shao et al., 2026), expert-rated technical feasibility is associated with realized coverage, while worker desire for automation has no detectable association with the outcomes examined.
4. **Location within occupations:** Across the 178 occupations for which within-occupation placement can be estimated, matched tasks lie near the midpoint of the expertise distribution on average. But, occupational groups differ around that mean. Published tools reach further toward specialized tasks in computing and in the small healthcare sample, while they are concentrated closer to routine tasks in production and in the small legal sample. A supplementary classification suggests that specialized work receives more tool coverage when it is already carried out through software.

The remainder of this paper is divided into 9 sections. Section 2 reviews and situates our study among related work, and Section 3 describes the construction of ATE. Sections 4 and 5 analyze direct matches and the broader taxonomy of agent tools. Sections 6 and 7 compare tool supply with exposure, technical feasibility, worker preferences, and task expertise. Section 8 discusses the implications and limits of this evidence, and Section 9 concludes.

Our results suggest that measuring agentic automation requires more than asking whether AI can perform existing human tasks, it requires observing which actions are made available to machines, where those actions fall within occupations, and how machine-callable work is decomposed and recombined. The division of labor between people and AI is not only shaped by which human tasks machines can perform, but by how work is redefined into units that machines can perform at all. ATE provides an initial snapshot of this emerging supply and an open resource for tracking how it evolves.

2 Background

2.1 The Public MCP Ecosystem

The Model Context Protocol (MCP) is an open standard for connecting language-model applications to external tools and data sources. When a users’ request requires current, private, or action-oriented information, an MCP-enabled application allows a model to access tools to complete the request from a connected server. The application and model select a relevant tool, pass structured arguments to it, and return the result to the model’s context. Public MCP directories list servers that connect to services, data sources, and local systems, often exposing several tools through a single server.

Consider the request to, *find a highly rated coffee shop nearby*. For a language-model, answering this request accurately requires a map and search tool with access to both current ratings and the user’s location. The model can call these tools with structured inputs, receive structured results, and use those results to compose its response. The external service supplies the situational information that is absent from the model’s parameters.

The MCP ecosystem has existed since late 2024 [Anthropic \(2024\)](#), and since then has greatly expanded the range of external actions that language-model applications can initiate. Depending on the connected servers and the surrounding application, a model can query a database, edit a file, open a pull request, schedule a meeting, or participate in a workflow that combines several such actions. The growth of this public MCP ecosystem makes developers’ tooling choices observable at scale, and ATE represents 123,069 of these distinct server listings from seven directories and 694,411 deduplicated tools, providing the largest public snapshot of the functions made available to AI systems.¹

2.2 Related Work

This study sits at the intersection of four literatures: (1) the measurement of AI’s exposure to work, (2) the task-based framework for reasoning about where automation lands inside a job, (3) the emerging use of bottom-up taxonomies to characterize AI-relevant tasks, and (4) the empirical study of the MCP ecosystem itself.

The first is the growing body of work on AI exposure measurement. [Eloundou et al. \(2023\)](#) estimate that large language models have substantial exposure to the tasks of roughly 80 percent of U.S. workers. [Felten et al. \(2021\)](#) link AI capability advances to occupation-level exposure. Alongside these sit demand-side measures that classify what users actually bring to AI systems, analyzing large volumes of conversations to infer which occupational tasks are being brought to a model and with what degree of automation [Handa et al. \(2025\)](#); [Tomlinson et al. \(2025\)](#); [Chatterji et al. \(2025\)](#); [Isenko et al. \(2026\)](#). Together, these measures describe modeled capability and observed user demand. Our supply-side measure rests on a different observation that publishing an MCP tool makes a defined function callable by an AI application. This signals developer interest in delegating that function, but it does not establish that the surrounding task can proceed without human oversight. This bounds the conclusions ATE can support on its own – a published tool records available supply, not deployment, reliability, adoption, or successful automation. ATE is a complementary piece of evidence to these other measures, and is a uniquely valuable signal for monitoring early trends ahead of deployment, and for isolating the types of tasks which are fully

¹ATE is a snapshot of public MCP directories only. We discuss what may be missing from private servers, and other qualifications on scope in Section 7.1.

automatable, start-to-finish.

The second literature is the expertise framework of [Autor & Thompson \(2025\)](#), who formalize the distinction between expertise-raising and expertise-lowering automation using the Standard Frequency Index (SFI) measure. This framework is utilized by [Teeselink & Carey \(2026\)](#) who provide quasi-experimental evidence, exploiting ChatGPT’s release across 39 countries, that expertise-raising automation predicts higher advertised wages while aggregate AI exposure predicts fewer job postings. We adopt and extend [Autor & Thompson](#)’s expertise framework to the MCP setting, adding a within-occupation percentile normalization that makes the expertise measure comparable across occupations.

The third is the emerging literature on bottom-up AI task taxonomies. The CLIMB framework ([Li et al., 2025](#)) builds an occupation taxonomy from the ground up, semantically clustering raw job postings and assembling the hierarchy with a generator-and-evaluator agent loop; Section 5.1 explains why our clustering departs from its exemplar-based step at our corpus size. The Anthropic Economic Index ([Handa et al., 2025](#)) uses a similar approach as a validation exercise. We apply a related bottom-up methodology to MCP tools in order to characterize the large share of the corpus that does not match O*NET tasks directly. This analysis asks which categories become visible when the taxonomy is derived from tool descriptions rather than imposed from an occupational database.

The fourth literature collects MCP artifacts directly. [Lin et al. \(2025\)](#) introduce MCPCorpus, roughly 14,000 servers and 300 clients, each annotated with more than twenty normalized attributes covering identity, interface configuration, and GitHub activity to provide an ecosystem snapshot. [Guo et al. \(2025\)](#) take a measurement approach, building MCPCrawler to normalize listings from six markets, from which 17,630 raw entries reduce to 8,401 valid projects (8,060 servers and 341 clients). [Wu et al. \(2025\)](#) assemble the largest prior corpus, MCPZoo, comprising 129,059 server listings of which 56,053 are distinct.

These prior MCP ecosystem studies are complementary to ATE. [Guo et al.](#)’s validity result motivates our own deduplication by source repository and our reporting of link-resolution rates; [Wu et al.](#)’s corpus is the benchmark against which we measure our coverage. Joining on normalized repository identifiers, ATE contains 85% of MCPZoo’s distinct servers, and holds roughly 66,000 servers it does not. Each of our seven source registries overlaps a nearly disjoint slice of MCPZoo, with the per-source intersections summing to almost exactly the total. The limited overlap indicates that the directories capture different parts of the public ecosystem, so aggregating across them substantially increases coverage.

The closest prior work to ours is [Stein \(2026\)](#), who evaluates 177,436 agent tools created between November 2024 and February 2026 by monitoring public MCP server repositories. Stein classifies each tool by its direct impact and uses an O*NET mapping to identify each tool’s task domain and corresponding consequentiality. Stein reports that software development accounts for 67% of all agent tools and 90% of server downloads, and that the share of action tools rose from 27% to 65% of usage across a sixteen-month window. While this work asks what kind of impact a tool has and how consequential the affected domain is, we ask whether a tool performs a recorded work task from end to end, filtering out tools that inform a human from tools that fully automate. Our main outcome is therefore the share of tools that match a recorded task from end to end rather than a distribution across impact classes. Like Stein, we also find a strong concentration of tools for computing occupations.

Relative to prior MCP datasets, ATE contributes a substantially larger tool-level corpus. It also introduces a stricter tool-task matching test, which achieves 76.7% precision in a blind human validation sample, and a within-occupation analysis of where matched tasks fall in each job’s expertise distribution. Finally, rather than discarding the roughly 97% of tools without a direct match, we use bottom-up clustering to characterize their structure.

3 Building The Agentic Task Ecosystem

3.1 Data collection

We collected server listings from seven public MCP directories in May 2026: the official MCP registry, Glama, MCP Store, MCP World, Smithery, MCP Repository, and mcp.so. For each listing we recorded the server’s name, description, linked source repository, and all listed tools’ names and descriptions. No single directory provides a comprehensive view of the public MCP ecosystem, and listings overlap only partially across sources. We therefore aggregate across directories before deduplicating by normalized source repository. Where a listing resolves to a GitHub repository we enriched it with repository metadata (stars, forks, language, license, creation and push dates); 90.1% of servers resolve (110,835 of 123,069; among the 118,751 with a GitHub url, 93.3%). Tool inventories are extracted from listing pages and README files; we recover at least one parseable tool for 42.1% of 123,069 server listings, yielding 696,291 tool listings before tool-level deduplication. The remaining 57.9% yield no parseable inventory. This does not imply that these servers expose no tools: missing inventories can arise from incomplete documentation, unsupported listing formats, or genuinely empty servers. Therefore ATE should be interpreted as a census of publicly observable tool descriptions within the collected directories, rather than a census of all functionality implemented by the listed servers.

3.2 From Tools to Tasks: Top-Down Matching

Our unit of observation is the tool: a named action with a natural-language description of what it does. We treat the tool as a unit of published functionality and not as a unit of human work, and determining whether and how these machine-defined units correspond to human occupational tasks is an empirical question rather than an assumption of our analysis. To connect tools to recorded work, we begin with 18,796 task statements across 923 occupations in O*NET 29.2. A language model (Command A Plus, prompt version `computer_task_filter_v5`, Appendix A.1) classified 9,423 of those statements as software-performable under a single criterion: whether the cognitive, digitizable core of the task could in principle be carried out by software. This filtering step is intended to define the relevant opportunity set by asking whether the task can be completed on a computer, and thus the filtered universe deliberately excludes physical and interpersonal work. All coverage rates in this paper are shares of software-performable work, not of work overall.

Using this constructed set of software-performable task statements, we embed every tool description and every task statement into a shared vector space with Cohere embed-multilingual-v3.0 (1,024 dimensions; tools embedded on their name and description, task statements on the occupation title and task text) and retrieve the five nearest task statements for each tool. Only the top-ranked pair enters adjudication to determine the match quality.

Adjudication is performed by a language-model judge (Command A Plus, under prompt version `v7_strict`, Appendix A.3), and the judge assigns one of three labels. First, a pair is labeled **good** only when the tool’s action directly corresponds to what the task statement names. A tool that

is narrower along the object or input dimension still qualifies, while one that only performs a step toward the task does not. Second, a pair is **partial** when the task is broader than the tool’s action and a person retains coordination, when the tool only retrieves information, or when the judge is uncertain. Third, a pair is **bad** when the domain is wrong outright.

After the adjudication, only good pairs are counted as true O*NET matches. Under this strict standard, 18,058 pairs are judged good, 591,826 partial, and 84,522 bad. In other words, we find that only 2.6% of the total tools represented in ATE map cleanly back to task statements as defined by O*NET.

To validate this, we drew a stratified sample of 120 pairs from the good-match set and labeled them without seeing the model judge’s decisions. The strata reflect the number of tools attached to the underlying task statement, and the validation sample yields an overall precision estimate of 76.7%, with errors concentrated among sparsely matched task statements. For statements with at least 100 matched tools, 16.7% of sampled pairs are incorrect; for statements with fewer than ten matched tools, 43.3% are incorrect. Restricting the calculation to human labels marked confident reduces those error rates to 6.5% and 23.1%, respectively. Applying the observed stratum-specific rates to the population produces an adjusted match-rate estimate of approximately 1.9% to 2.2%, compared with the unadjusted rate of 2.6%.

The validation error suggests that 2.6% overstates the match rate within the observed corpus. Incomplete coverage of private and poorly documented servers may operate in the opposite direction, but its magnitude is unknown. We therefore report the observed 2.6% rate together with the validation-adjusted range and the scope limitations described in Section 7.1.

3.3 Beyond O*NET: A Bottom-Up Taxonomy

The direct matching methodology described above in Section 3.2 relies on the assumption that task boundaries from O*NET map cleanly onto agent tools. The resulting low direct-match rate (2.6%) can therefore have at least two interpretations: either MCP tools may have little relationship to occupational work, or they may encode work at a different level of granularity than occupational task statements. To distinguish these possibilities we reverse the direction of analysis. Rather than beginning with a taxonomy of human work and asking where tools fit, we derive categories bottom-up from the tools themselves, only then asking how those categories relate to recorded occupational tasks.

To this end, we embed all 694,411 deduplicated tools with Cohere embed-multilingual-v3.0 (1,024 dimensions), on each tool’s name and description. The use of a multilingual model matches a corpus whose listings, while predominantly English, include tool text in dozens of languages. Clustering then runs on the full corpus rather than the unmatched remainder. This keeps the bottom-up representation independent of the top-down matching decision and allows us to compare the two views of work rather than defining one from the residual of the other. The resulting representation places semantically similar tool descriptions near one another, including descriptions written in different languages.

Two features of the corpus guide the clustering method. First, the number of meaningful tool categories is not known in advance. Second, category sizes and densities vary substantially, from thousands of near-duplicate tools to small groups of less common functions. The method must therefore infer the number of clusters, accommodate uneven densities, and allow some tools to remain unassigned. These requirements also rule out the exemplar-based clustering CLIMB uses

(Li et al., 2025): Affinity Propagation requires a full pairwise similarity matrix quadratic in corpus size, which is infeasible at 694,411 tools, and it assigns every point to a cluster rather than allowing refusal. LLM-guided clustering can produce higher-quality categories on smaller corpora but is prohibitively expensive at this scale; our pipeline reserves LLM effort for the labeling and classification stages that follow.

We therefore cluster with BERTopic (Grootendorst, 2022), which combines two steps suited to our corpus’ properties. UMAP (McInnes et al., 2018) first reduces the embeddings to five dimensions while preserving each tool’s local neighborhood (cosine metric, 15 neighbors, minimum distance 0, fixed seed); this step matters because the density estimates the clustering relies on become unreliable in very high-dimensional spaces. HDBSCAN (Campello et al., 2013) then forms clusters from regions where tools are dense, discovering the number of clusters from the data, tolerating clusters of different densities, and, importantly for our counts, declining to assign tools that sit in no dense region rather than forcing them into the nearest cluster. HDBSCAN leaves 306,624 tools (44.2%) outside any dense region. These observations are consistent with a long tail of uncommon or one-off tools; we retain them in the released dataset as noise but exclude them from category-level analysis. The one substantive parameter is the minimum cluster size, the smallest group of tools we are willing to call a category of work. We use 50, from a sensitivity sweep over 10, 25, 50, and 100, which yield 9,457, 3,607, 1,712, and 858 clusters respectively. The taxonomy’s category counts should be read with that granularity choice in mind. The noise share is stable across the sweep, which marks the unassigned tail as structural rather than an artifact of the threshold.

Finally, we classify the 1,712 clusters by the type of activity they describe. We exclude 573 clusters as non-occupational: 336 describe protocol infrastructure, 188 describe consumer activity, 36 cannot be characterized reliably, and 13 are incoherent. A language model (Command A Plus; prompts in Appendix A.5) labels the remaining 1,139 occupational clusters from their member tool names and descriptions. A second cluster-level judgment then classifies each cluster as subatomic, composite, or new from its description and its nearest software-performable task statements. **Subatomic** clusters represent the smaller components of recorded occupational tasks; for example, database schema extraction, one of the largest clusters at 1,983 tools, is a component of database administration that no single O*NET statement records at that level of detail. **Composite** clusters describe work that is not a sub-step of any single recorded task; the described activities span several tasks or bundle them into a workflow O*NET does not name (meeting-transcript management, for example, spans the collection, organization, and analysis of transcripts). **New** clusters have no identified occupational precedent in O*NET. The final taxonomy contains 693 subatomic clusters, 411 composite clusters, and 35 new clusters. Thematic grouping produces 28 subatomic themes containing 151,597 tools and 25 composite themes containing 108,854 tools.

4 Which Work Tasks Do Published Tools Perform?

As described in Sections 3.2 and 3.3, we construct ATE through two complementary methods. The top-down analysis embeds each tool against the software-performable O*NET universe and applies a language-model judge to determine whether the closest pair constitutes an end-to-end task match. The bottom-up analysis independently clusters the full tool corpus from tool descriptions without conditioning on O*NET matches, to recover categories of machine-callable activity at their native granularity.

The remainder of this section describes the types of work, and the corresponding occupations for which we observe tools under both methods. We start by characterizing the direct-matches – asking for which occupations the 2.6% of matched tools land. We then explore the clusters derived from

Grouped MCP tools concentrate in recognizable work and infrastructure

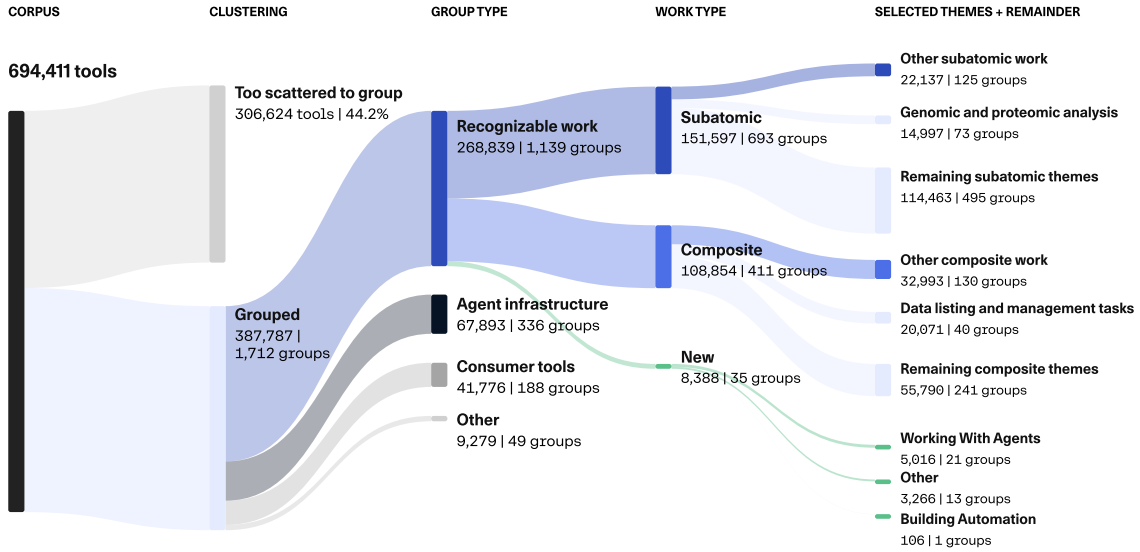


Figure 1: Clustering covers all 694,411 deduplicated tools, including those with judged good matches. The three-way work-type classification and its thematic layers: 693 subatomic and 411 composite clusters group into 28 and 25 themes (151,597 and 108,854 tools), with 35 new clusters (8,388 tools) alongside.

our bottom-up taxonomy – asking what kinds of work are represented in the subatomic, composite and new streams.

4.1 Direct Occupational Coverage

The share of tools with direct correspondence to O*NET task statements can be summarized at three levels: (1) tool-level matching asks what share of published tools directly execute a recorded task, (2) task-level coverage asks what share of recorded software-performable tasks have at least one corresponding tool, and (3) occupational coverage asks how broadly those matched tasks are distributed within and across occupations.

First, we observe that matched tools attach to 1,380 distinct task statements, roughly 15% of the software-performable work O*NET records. Direct correspondence is sparse across all occupations, and concentrated within those that do match – only about 15% of software-performable O*NET statements receive any direct match, and a small number of occupations account for a disproportionate share of matched tools. Across occupations, the ten most-covered account for approximately 44% of all matches.

For these occupations with direct coverage, the matched tools often pile onto one or two broadly worded statements rather than spreading evenly across its full task distribution. For example, Graphic Designers have tools matching 11 of their 15 software-performable tasks, but over a thousand of those tools attach to a single statement, “Use computer software to generate new images.” This is an example of a valid match while still inflating apparent occupational coverage due to a

MCP tool coverage is shallow across most occupations

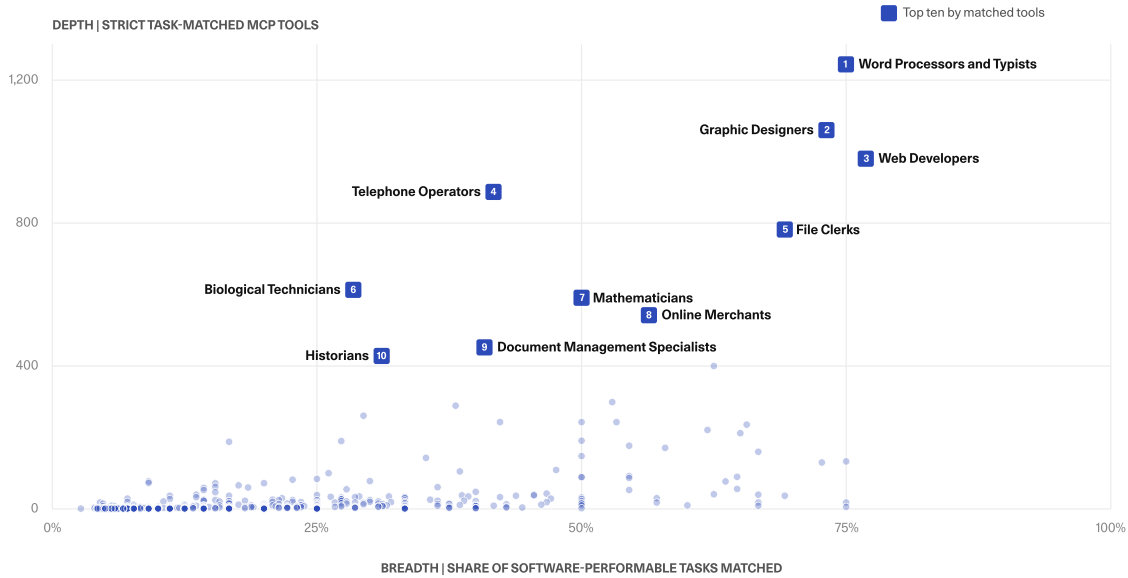


Figure 2: Breadth versus depth of MCP coverage across occupations. Of the 786 occupations meeting the coverage floor (at least three software-performable task statements), the 473 with at least one matched tool are shown (log scale). The remaining 313 occupations meet the floor but have no direct match.

broadly phrased task. To address this, we examine both the breadth and depth of tool coverage, as they capture different dimensions of supply (Figure 2). We also only report coverage for occupations with at least three O*NET task statements, which retains 786 of the 895 occupations represented. Without this floor, a single match can produce 100 percent coverage for an occupation represented by only one or two software-performable statements. Within the 895 occupations of the software-performable universe, 404 have no directly matched task at all, median direct coverage of an occupation’s software-performable tasks is 5.9%, and only 8 occupations reach full coverage.

Since direct matches only represents tools that perform a whole task, we also designed a broader measure in order to understand how much tool activity actually exists for a given occupation. This measure adds three groups of tools together. The first is tools with a judged good match. The second adds tools in subatomic clusters, assigned to whichever occupation their parent task belongs to. The third adds tools in composite and new-work clusters, assigned the same way. Each layer also includes everything below it, so the third is a cumulative total rather than an increment (Figure 1).

Of 923 O*NET occupations, 504 show activity at the broadest layer and 419 show none at any layer. Among the 504, ranking occupations by broad tool count gives much the same ordering as ranking them by direct matches (Spearman $\rho = 0.696$). The largest totals are Network and Computer Systems Administrators (28,678 tools), Database Administrators (22,566), Document Management Specialists (17,646), and Software Developers (14,670).

While direct matches make up only a small percentage of our overall corpus, this does not mean that the remaining tools are unrelated to existing work. In the following section, we examine whether these tools are operating at a different unit of action, asking not only how much of O*NET maps directly to MCP tools, but whether human task boundaries are the appropriate granularity for

describing machine work in the first place.

4.2 Subatomic Work

After clustering the unmatched tools as described in Section 3.3, we find that the majority of tools (693 of 1,139 clusters) *do* describe existing work, just at a finer grain. Where an O*NET statement records “maintain configuration control,” ATE holds separate tools for individual configuration reads, writes, validations, and rollbacks. Subatomic clusters describe familiar occupational work at a smaller unit than O*NET typically records. This granularity mismatch has a measurement consequence for supply-side coverage computed against O*NET statements, which will understate activity due to tools performing constituent parts of work not matching a statement that describes an end-to-end task. It also has an economic consequence as the subatomic layer records which parts of recorded tasks builders consider separable, and the separability of task bundles is what determines how work can be re-divided between people and machines as automation proceeds. This suggests that occupational task statements are not necessarily the atomic unit of agentic automation, as we see that a recorded human task can itself become a bundle of independently machine-callable operations.

4.3 Potentially New Machine-Callable Tasks

A small tail of clusters (35 of 1,139) describes work with no occupational precedent. The emerging agent ecosystem is therefore characterized much more by decomposition and recombination of existing work than by the creation of entirely novel categories. Twenty-one of the 35 new clusters concern work performed on or with agents, one concerns the construction of automation, and 13 do not fit either category. Examples of novel tasks include selecting synthetic voices, managing AI personas, and assessing agent trustworthiness. We read the rarity of new machine work as informative in its own right, and at this stage of the ecosystem, builders overwhelmingly automate and subdivide recognized work. Independent human labeling of 120 categories, conducted without access to the model’s classifications, identifies a similarly small share as new and produces a slightly lower estimate than the model. We caution that novelty here is judged against O*NET’s recording of work, and O*NET’s own coverage of recent work is incomplete (see Section 7.1).

4.4 Agent-Dependent Work

For each cluster we asked whether the work could exist without autonomous agents. Corpus-wide, only 29 of 1,139 clusters are judged agent-dependent, but they concentrate overwhelmingly in the new category: 18 of the 35 new clusters are agent-dependent, against 11 of the 1,104 subatomic and composite clusters. The three largest are Skill repository management (1,464 tools), Semantic search and retrieval (1,125), and Multi-agent orchestration management (811). Many agent-dependent clusters concern the governance of delegation: configuring what an agent may do (Agent configuration management, AI persona management), monitoring or verifying its actions (Agent trust and reputation assessment, Agent status monitoring and assessment), and intervening when it fails. This convergence, in which the newest machine work generates oversight work for humans, is consistent with the reinstatement mechanism the task literature describes (Acemoglu & Restrepo, 2019), though supply-side data cannot show whether anyone is yet employed to do it.

4.5 Is Agentic Development Targeting the Human-Centered Parts of Work?

A natural question to ask of a corpus describing automation is exactly what kinds of work are represented. Specifically, we are interested in whether developers are handing machines the most human-centered parts of work. To begin answering this, we rated each cluster’s description on the five EPOCH dimensions of human capability; empathy, presence, opinion, creativity, and hope (Loaiza & Rigobon, 2024).

We find that the kinds of work represented in ATE score low on every dimension. On a continuous scale of 1-4, the mean overall scores are 0.48 for subatomic clusters, 0.73 for composite clusters, and 0.78 for new clusters. According to the EPOCH measure, what developers are packaging for machines is, overwhelmingly, types of work that human capabilities are least needed for. The one capability machine work *does* touch is judgment, as the opinion dimension carries essentially all of our recorded signal. Within the subatomic and composite groups, the further a cluster is in vector space from a recorded O*NET task, the lower it’s human-capability rating. In other words, the less a cluster relates to existing work, the less human capability is required to perform it. For subatomic clusters, categories further from recorded human work rate lower on the capabilities (Spearman $\rho = -0.20$), and the same holds within composite clusters ($\rho = -0.17$; both significant).

There is one exception to this finding. Of the 35 clusters classified as genuinely new, 18 are agent-dependent, and these rate higher on opinion and creativity than any other group (mean 1.09). This specific kind of work is supervisory. For example, configuring how an agent behaves, evaluating what it produced, deciding whether it can be trusted are all judgment calls without determinate answers. The rest of the new category – building automations, moving data between systems – is procedural, and scores low on the human capability scale accordingly.

The overall picture is that almost no published tool supply is directed at work scoring highly on empathy, presence, creativity, or hope, and what little there is concerns the work of overseeing agents. Because ATE records published tools, none of this establishes what work actually remains with people or whether new demand for labor has emerged.

Two qualifications are central to this analysis. First, the ratings describe how tools and clusters are written about, not work as it is actually performed. Second, the corpus only contains functions developers chose to expose to AI systems, so low capability scores may reflect what ends up in the dataset as much as what the work itself demands. Composite work scores reliably higher than subatomic work. The difference is about 0.25 on the 0–4 scale – roughly half the subatomic mean – and it holds when we keep only the half of clusters whose ratings were most stable across runs. The gap between new and subatomic work does not survive that check ($p = 0.26$, on 35 clusters), and nothing above depends on it. Novelty is one minus the nearest-task similarity shipped with the dataset, so every figure here can be reproduced from the release.

5 What Predicts Where Tools Get Built

In the following sections we connect our tooling observations with theory in order to translate how these developments connect and may be translated to real world impact. We first ask what factors predict where this supply emerges – if published tooling simply follows technical capability, existing exposure measures may already provide a good approximation. If not, the gap between what AI could do and what developers choose to build becomes an object of study in its own right.

EPOCH ratings: thin engagement, judgment-led



Figure 3: Mean EPOCH-style capability ratings (0–4) of category descriptions, by agent dependence. Mean ratings are low across the five dimensions, with the largest values on the opinion dimension. The exception is agent-dependent categories, elevated on judgment and creativity (agent-dependent new versus all other categories, Mann-Whitney $p = 0.0007$).

5.1 Capability Versus Realized Tooling

Across the 178 occupations with sufficient coverage, the published [Eloundou et al.](#) β exposure measure is positively correlated with realized ATE coverage, defined here as the share of an occupation’s full O*NET task list with a matched tool (Spearman $\rho = 0.537$, $p < 10^{-13}$). The strength of this association is notable because the measures are constructed from different evidence, although both ultimately reference O*NET tasks.² Exposure is not significantly associated with the within-occupation expertise position of matched tasks ($\rho = 0.11$, $p = 0.15$). Coverage depth behaves the same way: neither tool counts ($\rho = 0.02$, $p = 0.77$) nor the share of tasks matched ($\rho = -0.08$, $p = 0.29$) predicts where matched tasks sit in the occupation’s own mix. Capability therefore appears more informative about whether an occupation attracts tooling than about which parts of that occupation tooling reaches. Knowing that a job is highly exposed, or even heavily tooled, does not tell us whether published tools concentrate on its relatively routine or specialized tasks.

5.2 Technical Feasibility Versus Worker Preference

Technical feasibility is one possible determinant of tool supply, whereas a worker-centered account would instead predict that tooling reflects which tasks workers want AI assistance with or delegation of. WORKBank ([Shao et al., 2026](#)) allows us to compare these two hypotheses directly for a subset of occupations. WORKBank records, for the tasks in its reference set, workers’ desire for automation and expert assessments of AI capability. Aggregating both to occupations and intersecting with our expertise sample (see Section 6) leaves 65 occupations carrying both ratings and at least three matched tasks. In this 65-occupation sample, expert-assessed capability is positively associated with realized coverage (Spearman $\rho = 0.31$, $p = 0.011$). Worker desire for automation is not significantly associated with coverage ($\rho = 0.09$, $p = 0.50$), the number of distinct matched servers ($\rho = -0.08$, $p = 0.50$), or the direction of expertise change ($\rho = -0.07$, $p = 0.60$). The two coefficients should not be interpreted as significantly different from one another: a Williams test of the dependent correlations does not reject equality ($t = 1.36$, $p = 0.18$), so we do not claim that

²The coverage denominator here is the occupation’s full O*NET task list rather than the software-performable subset; using the software-performable subset as the denominator yields $\rho \approx 0.21$. We use the full-task denominator for comparability with the published exposure measure and report this sensitivity result here.

Higher AI task exposure is associated with broader MCP coverage

Eloundou task-share exposure compared with the share of O*NET tasks matched across 178 occupations.

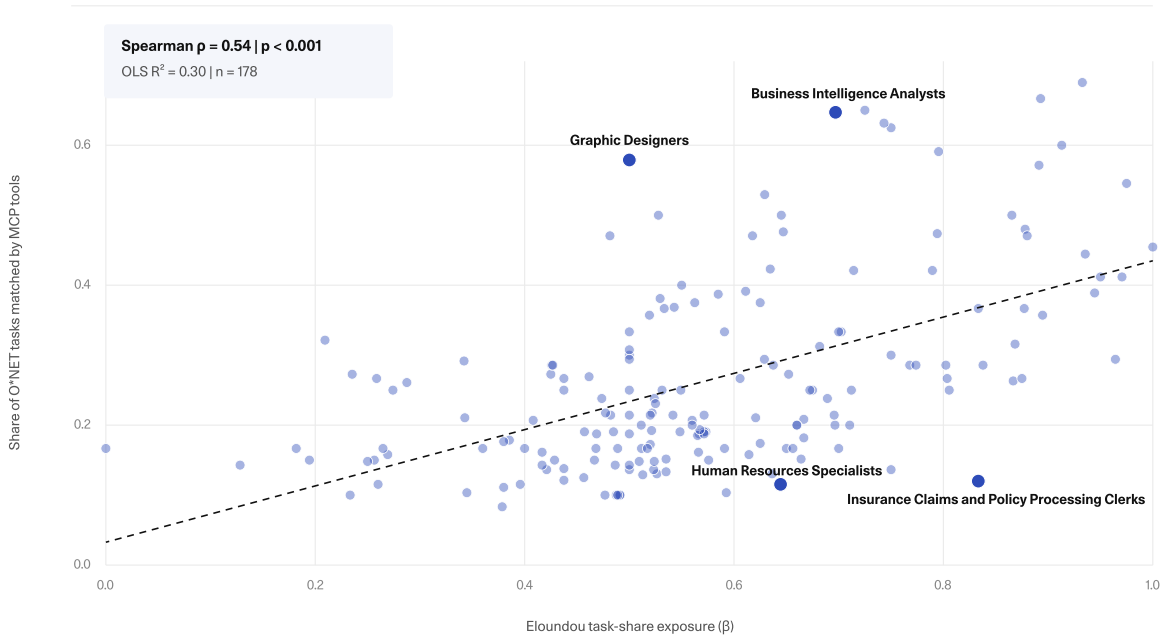


Figure 4: Theoretical exposure versus realized coverage across the 178-occupation sample (Spearman $\rho = 0.537$). Coverage is the share of an occupation’s full O*NET task list with a matched tool. Labeled occupations are the largest verified residuals in each direction.

capability predicts coverage significantly more than desire does. The evidence therefore supports a positive association between expert-rated capability and coverage in this sample, while providing no statistically detectable association for worker desire (Figure 5). The null results do not establish that worker preferences have no influence; they show that any such influence is not visible in these occupation-level measures and this sample.

Published supply is ultimately the outcome of several selection processes including technical feasibility, developer and customer demand, commercial incentives, data access, and platform constraint, only some of which we can observe here.

5.3 Where Capability and Supply Diverge

We observe two forms of divergence between theoretical exposure and published tool supply. Some occupations receive high theoretical exposure scores but little or no observed public tooling, while others receive substantially more tooling than their exposure scores would predict. If capability and tool supply capture distinct stages of automation, their disagreements may be particularly informative. In this section we examine the occupations for which the divergence between exposure and supply is greatest.

To identify these occupations, we fit a simple regression of realized coverage on theoretical exposure across the 178-occupation sample ($R^2 = 0.30$). A positive residual means an occupation has more tooling than its exposure score predicts while a negative residual means it has less. The largest positive residuals include Graphic Designers, Business Intelligence Analysts, and Word Processors and Typists. The largest negative residuals include Insurance Claims and Policy Processing Clerks

Coverage rises with assessed capability, not detectably with worker desire

The same 65 occupations and realized-coverage outcome are shown in both panels.



Figure 5: Worker automation desire versus distinct matched MCP servers, for the 65 occupations with WORKBank ratings: $\rho = -0.08$ ($p = 0.50$); expert capability versus coverage is $\rho = 0.31$ ($p = 0.011$). The Williams test does not reject equality of the two associations.

and Human Resources Specialists.

These divergences are the cases in which capability alone is least sufficient to explain the observed supply of tools. Exposure without observed supply can reflect unmet technical, commercial, or data-access constraints, as well as activity outside the public directories we observe. Supply beyond predicted exposure can reflect tool development overlooked by the exposure measure, broadly worded O*NET statements, or limitations in our matching procedure. Such cases provide natural targets for studying the institutional, commercial, and technical forces that mediate the path from AI capability to workplace automation.

Taken together, the analyses across Section 5 suggest that technical capability signals but does not determine the emerging supply of agent tools. Exposure is associated with how much occupational tooling we observe, but neither exposure nor tool volume predicts where within the expertise structure of a job that tooling falls. Worker preferences provide no detectable additional signal in the smaller sample we can study, although the evidence is insufficient to conclude that preferences matter less than feasibility. The path from capability to automation is therefore selective and what becomes machine-callable reflects choices and constraints not captured by capability alone.

6 Near-zero Average Hides Opposing Expertise Shifts

Occupational coverage measures how broadly agent tools reach into a job, but treats all tasks within that job as equivalent. In reality, work is not exposed to automation evenly and in order to predict outcomes for workers, we need to understand for what specific tasks we see tool matches for. Automating a relatively routine task may leave a more expertise-intensive bundle of human work, while automating a relatively specialized task may remove part of the occupation's expert

core. We therefore ask not only how much of an occupation attracts tool supply, but *which part of its expertise distribution* those tools reach.

6.1 Expertise Shifts

For each occupation in our sample, we place the matched tasks within the occupation’s own distribution of task expertise, using the word-frequency expertise measure of [Autor & Thompson \(2025\)](#) normalized to within-occupation percentiles, and summarize the occupation by its mean matched percentile. This normalization makes positions comparable across occupations and avoids the sign conventions of change-scores. Averaged across all 178 occupations, matched tasks sit almost exactly at the midpoint of occupations’ own expertise ranges: on average, tools are neither notably routine-seeking nor notably expertise-seeking.

However, occupational-group averages differ around the pooled midpoint. In healthcare practitioners ($n = 5$) and computer and mathematical occupations ($n = 30$), matched tasks fall above the midpoint on average. Under Autor and Thompson’s framework ([Autor & Thompson, 2025](#)), this pattern is consistent with expertise-lowering automation as tooling reaches relatively specialized tasks. In production ($n = 12$) and legal occupations ($n = 4$), matched tasks fall below the midpoint on average, a pattern consistent with expertise-raising automation as tooling is concentrated in relatively routine tasks. These estimates describe group means, and the healthcare and legal results in particular rest on small samples. Individual occupations within each group vary, so we do not assign a direction to any single occupation from its group membership.

Two task-level examples show how an occupation’s percentile arises from which of its tasks get matched. For Legal Secretaries and Administrative Assistants, the matched task “Review legal publications and perform database searches to identify laws and court decisions relevant to pending cases” sits at the 36th percentile of the occupation’s own expertise mix (a semantic legal-text search tool matches it), while “Prepare, proofread, or process legal documents, such as summonses, subpoenas, complaints, appeals, motions, or pretrial agreements,” at the 100th percentile, has no match of any quality. In this example, the matched tool covers information retrieval while the highest-percentile document-preparation task has no match. For Medical Records Specialists the pattern inverts – a transcription tool matches “Transcribe medical reports,” that occupation’s 100th-percentile task. This example illustrates a match at the specialized end of one healthcare occupation’s task distribution.

6.2 Specialized Work Is Reached When It Runs on Software

The SOC groups whose tools reach furthest into specialized work are not obviously alike as occupations, but they share what they do with information. Classifying the 178 occupations on that basis, rather than by SOC, separates them more cleanly. We classify the 178 occupations by a title-based rule into three classes: (1) domain-embedded information work, whose object is one professional domain’s data ($n = 12$; clinical data, bioinformatics, GIS, medical records); (2) general information infrastructure with no domain object ($n = 22$; database architects, security, document management); and (3) domain service, the residual that delivers a field’s primary service ($n = 144$). Mean matched percentiles are 61.4 for domain-embedded information work, 55.5 for general information infrastructure, and 49.6 for domain service (Kruskal-Wallis $H = 11.8$, $p = 0.003$). The association for domain-embedded information work remains after adjusting for tool counts (partial Spearman $\rho = 0.20$, $p = 0.007$). In this classification, published tools reach further toward specialized tasks when expert judgment is exercised through software and less when the specialized work is primarily physical or interpersonal. Computing rests on thirty occupations and is the best-evidenced case;

SOC-group averages sit within broad occupational variation

Large markers summarize SOC groups. Points in the background show the underlying occupations.

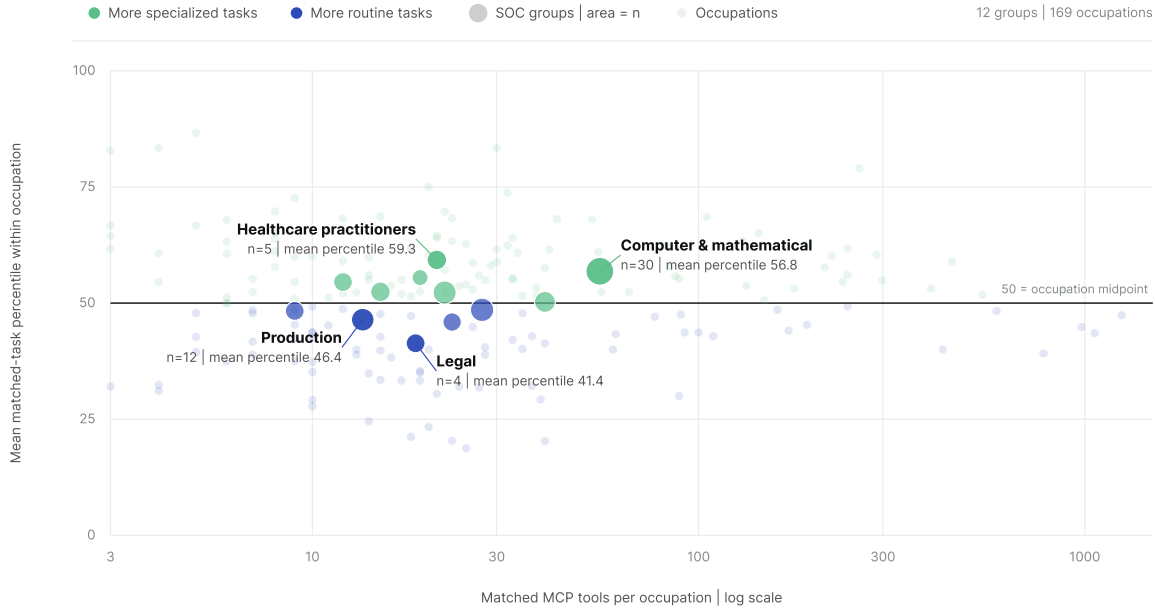


Figure 6: Mean matched percentile by SOC major group (groups with at least three sample occupations; 169 of 178 shown). Healthcare practitioners (59.3, $n = 5$) and computing (56.8, $n = 30$) sit above the midpoint; production (46.4, $n = 12$) and legal (41.4, $n = 4$) below it. Small-group means rest on few occupations.

healthcare practitioners sit further in the same direction but on only five occupations, which we treat as suggestive. These results suggest that specialized work receives more public agent-tool supply when it is already carried out through software. Specialized physical and interpersonal work receives less supply in the observed ecosystem. Whether that difference reflects technical feasibility, commercial incentives, access constraints, or adoption cannot be determined from ATE alone.

Together with Section 5.1, this reveals an important asymmetry. Theoretical exposure is informative about how much published tool coverage an occupation attracts, but neither exposure nor the amount of tooling tells us reliably which end of the occupation’s expertise distribution that tooling reaches.

7 Discussion

Our results suggest that the emerging supply of agent tools cannot be understood simply as the realization of model capability against a fixed set of human tasks. Capability is associated with the amount of tooling, but does not tell us which parts of work it targets, and machine-callable functions frequently cross the boundaries of recorded occupational tasks. In sum, the amount of tooling an occupation attracts is largely uninformative about where that tooling falls within its expertise distribution. These findings point to a more selective and compositional process through which AI technology becomes available for workers.

7.1 What ATE Measures and What it Cannot

ATE is a measure of supply, not adoption and not impact. A listed tool is a dated, public record that a developer exposed a function to an AI system, and the absence of tools means only that our collection contains no corresponding published record. Three main limitations bound the claims in this study. First is scope: we observe public directories only. The principal unobserved tier is the set of bespoke internal servers connected to private systems. While we do not have insight into private MCPs, vendor-published enterprise servers do appear in the corpus, so the omission is narrower than enterprise software as a whole. If those servers disproportionately target the back-office work represented in occupational databases, their omission would bias coverage rates downward, however we cannot estimate the size of that bias from the public data. Second is extraction: tool inventories are parsed from listings and READMEs, and the 57.9% of server listings with no parsed tools likely reflects a mixture of incomplete documentation, unsupported listing formats, and genuinely absent tool inventories. Servers with parsed tools have a median README length of 6,240 characters against 2,889 for zero-tool servers, and the zero-tool share varies with listing format by source, from 26% on Glama to 85% on Smithery. Third is our reliance on LLM as judge: the language-model judge has an estimated precision of 76.7%, with lower precision among sparsely matched statements. Applying validation rates produces an adjusted match-rate estimate of 1.9% to 2.2%, below the observed 2.6%. The matching universe also inherits O*NET’s lag in recording recent work. We retain analyst-sourced statements, which are often recent and lack incumbent frequency ratings, in the software-performable universe (990 of 9,423 statements, or 10.5%, compared with 8.7% of all statements). This choice avoids an additional exclusion of newer statements but cannot correct for work that O*NET has not yet recorded.

7.2 Implications

The expertise analysis connects the supply measure to a framework that links the location of automation within occupations to labor-market outcomes. Under that framework, automating relatively routine tasks can increase the expertise required of remaining workers, while automating relatively specialized tasks can lower barriers to performing the remaining work (Autor & Thompson, 2025; Teeselink & Carey, 2026). Prior evidence associates these configurations with differences in wages and entry, but ATE does not observe those outcomes directly. Further, neither configuration is plainly good or bad for workers. Automation which raises the required expertise may increase the wages of those workers, but limit the pool of qualified applicants. Automation which lowers the barrier to entry for an occupation can be viewed as democratizing expertise (Autor, 2024), but this doesn’t account for how skills and expertise are shaped by experience. Entry-level workers often learn through the routine tasks experienced colleagues and mentors hand-off. It’s already been observed that the hiring of young, entry-level workers in highly AI-exposed occupations has fallen well behind their peers’, and this gap has only widened over the past year (Brynjolfsson et al., 2026). If that work is increasingly done by machines, how people become experts strikes us as one of the more important open questions about AI and work. Overall, measuring the effects of both forms of automation requires additional data on adoption and worker-outcome. Our contribution is to identify where the observed supply of public tools is consistent with each configuration before adoption can be measured. Because listings are dated, future collections can track changes in the composition of public tool supply, and we are releasing ATE in order to support replication, extension, and comparison with subsequent snapshots.

8 Conclusion

In this paper, we introduced the Agentic Task Ecosystem (ATE), a large open corpus of public MCP tools linked to O*NET work tasks. The dataset adds a supply-side measure to existing evidence on modeled AI capability and observed AI use. Four findings characterize this May 2026 snapshot. First, direct end-to-end matches to recorded work are uncommon and concentrated: 18,058 of 694,411 deduplicated tools receive a good match under our judging standard. Second, bottom-up clustering shows that much of the ecosystem consists of components and combinations of recognized work, alongside agent infrastructure and a small set of categories with no identified occupational precedent. Third, theoretical exposure and expert-rated technical feasibility are associated with where tools appear, while worker desire is not. Fourth, the number of tools associated with an occupation does not indicate whether they reach its routine or specialized tasks. Group-level differences instead suggest greater reach into specialized work when that work is already mediated by software. The taxonomy’s structure carries a methodological implication for exposure measurement – human work is recorded in task statements, while supply arrives as components, bundles, and workflows, so task-level scores blur the margin of automation, and the reallocation of work within a single task is invisible to them. These findings describe published supply rather than adoption or impact. Repeated measurement, combined with data on deployment and worker outcomes, can test whether the patterns documented here persist as the ecosystem develops.

References

- Daron Acemoglu and Pascual Restrepo. Automation and New Tasks: How Technology Displaces and Reinstates Labor. *Journal of Economic Perspectives*, 33(2):3–30, May 2019. ISSN 0895-3309. doi: 10.1257/jep.33.2.3.
- Anthropic. Introducing the Model Context Protocol. <https://www.anthropic.com/news/model-context-protocol>, November 2024.
- David Autor. Applying AI to Rebuild Middle Class Jobs, February 2024.
- David Autor and Neil Thompson. Expertise, June 2025.
- David Autor, Caroline Chin, Anna M. Salomons, and Bryan Seegmiller. New Frontiers: The Origins and Content of New Work, 1940–2018, August 2022.
- Erik Brynjolfsson, Bharat Chandar, and Ruyu Chen. Canaries in the Coal Mine? Six Facts about the Recent Employment Effects of Artificial Intelligence, August 2026.
- Ricardo J. G. B. Campello, Davoud Moulavi, and Jörg Sander. Density-based clustering based on hierarchical density estimates. In *Advances in Knowledge Discovery and Data Mining (PAKDD 2013)*, pp. 160–172. Springer, 2013.
- Aaron Chatterji, Thomas Cunningham, David J. Deming, Zoe Hitzig, Christopher Ong, Carl Yan Shan, and Kevin Wadman. How People Use ChatGPT, September 2025.
- Tyna Eloundou, Sam Manning, Pamela Mishkin, and Daniel Rock. GPTs are GPTs: An Early Look at the Labor Market Impact Potential of Large Language Models, August 2023.
- Edward Felten, Manav Raj, and Robert Seamans. Occupational, industry, and geographic exposure to artificial intelligence: A novel dataset and its potential uses. <https://sms.onlinelibrary.wiley.com/doi/10.1002/smj.3286>, April 2021.

- Maarten Grootendorst. Bertopic: Neural topic modeling with a class-based tf-idf procedure. *arXiv preprint arXiv:2203.05794*, 2022.
- Hechuan Guo, Yongle Hao, Yue Zhang, Minghui Xu, Peizhuo Lv, Jiezhi Chen, and Xiuzhen Cheng. A Measurement Study of Model Context Protocol Ecosystem, November 2025.
- Kunal Handa, Alex Tamkin, Miles McCain, Saffron Huang, Esin Durmus, Sarah Heck, Jared Mueller, Jerry Hong, Stuart Ritchie, Tim Belonax, Kevin K. Troy, Dario Amodei, Jared Kaplan, Jack Clark, and Deep Ganguli. Which Economic Tasks are Performed with AI? Evidence from Millions of Claude Conversations, February 2025.
- Zanna Iscenko, Scott Strand, Yiyuan Chen, Guillaume Aimard, Mihai Codreanu, Vivek Sampathkumar, Alex Imas, Julian Jacobs, Evalyne Muiruri, Juan Mateos-Garcia, Jia Jen Ng, Samirah Javed, Josh Martin, Omar Ajmeri, Denis Calin, Andrew Kim, Fabien Curto Millet, and James Manyika. Google’s AI & Economy ATLAS v1.0: Mapping Gemini Usage in the Economy. 2026.
- Nan Li, Bo Kang, and Tijl De Bie. Building data-driven occupation taxonomies: A bottom-up multi-stage approach via semantic clustering and multi-agent collaboration. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pp. 1596–1614, Suzhou, China, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.emnlp-industry.113. URL <https://aclanthology.org/2025.emnlp-industry.113/>.
- Zhiwei Lin, Bonan Ruan, Jiahao Liu, and Weibo Zhao. A Large-Scale Evolvable Dataset for Model Context Protocol Ecosystem and Security Analysis, June 2025.
- Isabella Loaiza and Roberto Rigobon. The EPOCH of AI: Human-machine complementarities at work, 2024. URL <https://www.ssrn.com/abstract=5028371>. SSRN Working Paper 5028371.
- Campbell Lund, Thomas Euyang, Zanele Munyikwa, and Marzieh Fadaee. AI Exposure Scores: What they measure, what they miss, and what comes next, June 2026.
- Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.
- Yijia Shao, Humishka Zope, Yucheng Jiang, Jiaxin Pei, David Nguyen, Erik Brynjolfsson, and Diyi Yang. Future of Work with AI Agents: Auditing Automation and Augmentation Potential across the U.S. Workforce, February 2026.
- Merlin Stein. How are AI agents used? Evidence from 177,000 MCP tools, March 2026.
- Bouke Klein Teeselink and Daniel Carey. AI, Automation, and Expertise, January 2026.
- Kiran Tomlinson, Sonia Jaffe, Will Wang, Scott Counts, and Siddharth Suri. Working with AI: Measuring the Applicability of Generative AI to Occupations, December 2025.
- Mengying Wu, Pei Chen, Geng Hong, Baichao An, Jinsong Chen, Binwang Wan, Xudong Pan, Jiarun Dai, and Min Yang. MCPZoo: A Large-Scale Dataset of Runnable Model Context Protocol Servers for AI Agent, December 2025.

A Reproducibility: models, prompts, and run configurations

Generated from production artifacts at git commit `a0174578cc10dcdd4b0873df9f3744104830c77a`. All prompts below are reproduced verbatim from the files and run records of the production pipeline; tuning-round and draft prompts are excluded.

A.1 Software-performable task filter

Config verified in `outputs/filter_model_check.md`. Production run is task-filter **v5 Step 3** (full-universe score), not Campbell validation rounds 1–2.

Model	mcp-judge-ca-plus (Command A Plus, Model Vault)
Prompt version	computer_task_filter_v5
Prompt file	prompts/computer_task_filter_v5.txt
Decode	$T=0$, max_tokens=512, thinking_budget=256
Run report	outputs/task_filter_v5.md
Full labels	data/processed/task_filter_prompt_v5/all_tasks_labels_v5.jsonl
Shipped Yes set	data/processed/onet_tasks_computer_v5.jsonl (9,423 of 18,796)
Package mirror	data/share/mcp_corpus_v5_internal/tasks.csv (filter_* columns)

Prompt (verbatim):

Could software in principle perform the cognitive, digitizable core of this task, even if people usually do it themselves today?

Decision rules:

1. These are No. Work requiring physical action, physical presence, or in-person help from a person. Meetings and conversations with people. Supervising, training, or directing people. Coordination by voice, phone, or radio.
2. These are Yes. Decisions, approvals and authorisations issued through a system. Analysis, modelling, and evaluation of information. Acquiring, querying, entering or moving data. Producing documents, records and reports.
3. It does not matter whether people do it by hand today. Entering information into a system is Yes, because the activity is digitizable, whoever currently performs it. The question is whether software could in principle do it, not whether it already does.
4. Compound tasks. Some tasks describe more than one activity, and how they are joined decides the answer.

Activities joined by "and" must all qualify. "Develop and test surgical techniques" is No, because testing is physical.

Activities joined by "or" are alternatives, so answer Yes if any one of them qualifies. "Create sketches by hand or computer software" is Yes.

Items introduced by "such as" are illustrations, not requirements. Judge the activity they illustrate. "Perform clerical duties, such as answering phones, restocking supplies, and filling out paperwork" is Yes, because the activity is clerical work and the list is examples of it.

5. A computer inside a machine is not software work. When a computer appears as part of equipment, or as a physical object being installed, repaired or handled, the task is No. Operating a computer numerically controlled lathe is No. So is maintaining hand-held citation computers as equipment.
6. Do not infer from the occupation. The occupation is context only. A task is not physical merely because the occupation is a manual or field one, and not digitizable merely because the occupation is a technical one.
7. Ignore whether a tool for this exists. The question is whether the activity is the kind of thing software can perform, not whether anyone has built something that does it. Tool availability is judged separately,

later in the pipeline.

Examples:

Occupation: Teaching Assistants, Special Education

Task: Provide students with disabilities with assistive devices, supportive technology, and assistance accessing facilities, such as restrooms.

Answer: No

Occupation: Special Education Teachers, Elementary School

Task: Plan or supervise experiential learning activities, such as class projects, field trips, demonstrations, or visits by guest speakers.

Answer: No

Occupation: Rail Yard Engineers, Dinkey Operators, and Hostlers

Task: Confer with conductors and other workers via radiotelephones or computers to exchange switching information.

Answer: No

Occupation: Computer Numerically Controlled Tool Operators

Task: Set up and operate computer-controlled machines or robots to perform one or more machine functions on metal or plastic workpieces.

Answer: No

Occupation: Pediatric Surgeons

Task: Conduct research to develop and test surgical techniques that can improve operating procedures and outcomes.

Answer: No

Occupation: Medical Records Specialists

Task: Enter patient, specimen, insurance, or billing information into computer.

Answer: Yes

Occupation: Cardiologists

Task: Prescribe heart medication to treat or prevent heart problems.

Answer: Yes

Occupation: Financial and Investment Analysts

Task: Employ financial models to develop solutions to financial problems or to assess the financial or capital impact of transactions.

Answer: Yes

Occupation: Penetration Testers

Task: Evaluate vulnerability assessments of local computing environments, networks, infrastructures, or enclave boundaries.

Answer: Yes

Occupation: Electrical and Electronic Equipment Assemblers

Task: Complete, review, or maintain production, time, or component waste reports.

Answer: Yes

Now answer for this case. Answer format: "Yes" or "No"

Occupation: {occupation}

Task: {task}

A.2 Embeddings

Config verified in outputs/report_cleanup_lookups_2.md and outputs/clustering_config_check.md. No generative prompt.

Model	Cohere embed-multilingual-v3.0
Dimensions	1024
input_type	search_document
Tool embed text	{tool_name}: {description} (whitespace strip only)
Task embed text	{occupation title}: {task text}
Tool vectors	data/embeddings/tools_float32.npy (694,411 × 1024)
Task vectors	data/embeddings/tasks_float32.npy (18,796 × 1024)
Meta	data/embeddings/tools_meta.jsonl, data/embeddings/tasks_meta.jsonl

A.3 Match judge

Config verified in outputs/report_cleanup_lookups_2.md. Prompt file is bit-identical to PROMPT_CONFIGS["granularity_v7_strict"] in scripts/match_llm_judge.py.

Model	mcp-judge-ca-plus (Command A Plus, Model Vault)
Prompt version	granularity_v7_strict
Prompt file	prompts/granularity_v7_strict.txt
Script mirror	scripts/match_llm_judge.py (PROMPT_CONFIGS)
Full gate-free batch state	data/processed/match_llm_judge_granularity_v7_strict_full_batch_state
Computer_v5 rematch state	data/processed/match_llm_judge_v7_strict_computer_v5_batch_state.json
Package columns	data/share/mcp_corpus_v5_internal/onet_matches.csv (llm_model, llm_pro

Prompt (verbatim; <<<SYSTEM>>>/<<<USER>>> delimiters as in the file):

<<<SYSTEM>>>

You judge whether an MCP tool acts as the functional agent for an O*NET occupational task -meaning it can substantially execute or automate the task, not merely provide data, context, or reference information a human would use while performing the task themselves. Credit a tool as good if its specific action directly corresponds to what the task description names -even if the tool is narrower than the broadest possible reading of the task, the scope must match (e.g., a block-moving tool for "reformat documents, moving paragraphs or columns" is a scope match; a form-filling tool for "complete customs paperwork" is a scope mismatch because the task is broader than the tool's action). Do not credit a tool as good if: (1) the task description is broader than the tool's scope and the human still owns the coordination, reasoning, and completion of the task while the tool handles only one component; (2) its connection to the task is retrieval-only -it looks up or surfaces information a human then acts on; (3) it is a consumer-facing, simulation, or generic productivity tool that shares domain vocabulary with a professional task but does not operate in the actual professional context; or (4) its only connection is shared vocabulary or generic capability with no task-specific execution. When genuinely uncertain, default to partial rather than good. Respond with valid JSON only, no markdown fences.

<<<USER>>>

Judge whether this MCP tool acts as the agent executing this O*NET occupational task -specifically, whether the tool's action directly matches the scope of the task description, or whether the task is broader and the tool only handles one component of it.

Tool:

- name: {tool_name}
- description: {tool_description}

Matched O*NET task (rank {rank} for this tool):

- occupation: {occupation}
- task: {task_text}

Use this bar: "Does the tool's specific action directly correspond to what the task description names -or does the task describe something broader that the tool only partially addresses while the human still coordinates the rest?"

Scope-match test (apply first):

- If the tool's action directly matches the verb and object of the task description, count as good -even if the tool is a narrow, purpose-built capability. (Examples: block-moving tool →"reformat documents, moving paragraphs or columns"; calendar event creation →"schedule and make appointments"; SQL execution →"query databases to obtain information"; file conversion →"convert files for printing or the Internet.")
- A tool may be narrower than the broadest reading of the task and still count as good -provided it is narrower along the object/input dimension (one file format, one record type, one appointment type), not the process/step dimension. A tool that performs the task's named action on a narrower target is a scope match. A tool whose action is a step toward completing the task -rather than a constrained version of performing it -is not. (Example: convert_html_to_pdf performs the named action "convert files" on a narrower input type →good. puppeteer_fill performs form-filling, which is one step in the process of completing customs paperwork, not a narrower version of "completing paperwork" →partial.)
- If the task description is broader than the tool's action and the human must still coordinate, reason through, or complete the larger task, count as partial. (Examples: form-filling automation →"complete customs paperwork"; listing GitHub issues →"monitor or track project milestones and deliverables"; retrieving flight status →"determine timing and procedures for flight vector changes.")

Additional rules:

- A tool that retrieves relevant information (data lookups, status checks, record reads) that a human then acts on does NOT count as good; label it partial -unless the task itself is specifically a retrieval task (e.g., "search electronic sources for information").
- A consumer-facing, hobbyist, or simulation tool that approximates professional domain vocabulary but does not operate in the actual professional context does NOT count as good; label it partial or bad.
- A generic productivity tool (personal note-taking, generic spreadsheets, generic project management) matched to a professional operational task does NOT count as good unless the task specifically names that type of tool operation; label it partial.
- A tool whose only connection to the task is shared vocabulary or a generic capability (search, list, send, create, manage, file) with no task-specific execution does NOT count as good; label it partial.
- If genuinely unsure whether the scope matches, label it partial, not good.

Labels (use exactly one):

- good: the tool's specific action directly matches the scope of the task description -the tool executes what the task names, even if narrower in implementation along the object/input dimension
- partial: the task is broader than the tool's action and the human still owns coordination and completion; OR the tool's action is a step toward the task rather than a version of performing it; OR retrieval-only; OR consumer/simulation/generic tool not in the professional context; OR genuine uncertainty
- bad: wrong domain or occupation; no plausible way the tool supports this work

Examples:

- good: execute_sql →"Query databases to obtain information" (DBA): tool executes the named action directly
- good: calendar_create_event →"Schedule and make appointments" (Administrative): tool performs exactly what the task names
- good: convert_html_to_pdf →"Convert files for printing or the Internet" (Desktop Publishers): converts one specific file format -narrower along the input dimension, but the named action (file conversion for printing) is a direct match
- good: move_block →"Reformat documents, moving paragraphs or columns" (Word Processors): task names moving paragraphs; tool moves blocks -scope match
- good: generate_tests →"Develop testing routines and procedures" (Web Administrators): tool generates test code, which is what developing testing routines means
- good: API-patch-page →"Perform or direct Web site updates" (Web Developers): patching a page is performing a site update -scope match
- good: create-manual-journal →"Prepare adjusting journal entries" (Accountants): tool creates journal entries, which is what the task names
- partial: puppeteer_fill (web form automation) →"Complete customs paperwork" (Freight Forwarders): form-filling is a step toward completing paperwork, not a narrower version of completing it -human still gathers documents, verifies accuracy, coordinates submission
- partial: github →"Monitor or track project milestones and deliverables" (IT Project Managers): task is broader than listing issues; human still owns tracking, escalation, and coordination
- partial: get_flight_status →"Determine timing/procedures for flight vector changes" (ATC): retrieves delay data; does not execute ATC vector-change decisions
- partial: append_to_daily_note (Obsidian) →"Complete daily activity reports and keep records of messages from aircraft" (ATC): personal productivity app, not ATC operational logging
- partial: search_web →"Research competitor pricing strategies": generic search, no pricing-specific execution
- bad: engage_landing_autopilot (MSFS flight simulator) →"Transfer control of departing flights to traffic control centers" (ATC): video game simulation, not real ATC infrastructure
- bad: API docs lookup tool →"Enter text on keyboard for printed materials" (desktop publishing)

Respond with JSON only:

```
{{"match_quality": "good|partial|bad", "notes": "optional brief comment"}}
```

A.4 Clustering

Config verified in `outputs/clustering_config_check.md`. No generative prompt; embeddings as in §A.2.

Algorithm	BERTopic with precomputed UMAP + HDBSCAN
UMAP	<code>n_components=5, n_neighbors=15, min_dist=0.0, metric=cosine, seed=42</code>
HDBSCAN	<code>min_cluster_size=50 (primary); min_samples unset (=50); euclidean; eom</code>
Sensitivity sweep	<code>10 / 25 / 50 / 100</code> → 9,457 / 3,607 / 1,712 / 858 clusters
Noise	306,624 / 694,411 flagged and retained (not dropped)
Primary artifacts	<code>outputs/bertopic_results/ (run_meta.json, tool_clusters.csv, sensitivity_min_clu</code>

A.5 Cluster labeling

Production labels: `outputs/bertopic_results/cluster_labels_v6.csv` (manifest records model `command-a-plus-05-2026` on Model Vault). The 10–20 word O*NET-style constraint and expanded non-occupational exclusions live in the production **label** stage prompts below (v6 reuses the v5 label Gen/Eval files; gate is §A.6). Companion label evaluator/adjudicator prompts (same run, not printed in full): `prompts/step2_v5_label_evaluator_*.txt`, `prompts/step2_v5_label_el_adjudicator_*.txt`.

Model	<code>command-a-plus-05-2026 (Model Vault)</code>
Output	<code>outputs/bertopic_results/cluster_labels_v6.csv</code>
Manifest	<code>outputs/bertopic_results/cluster_labels_v6.manifest.json</code>
Label generator	<code>prompts/step2_v5_label_generator_system.txt, step2_v5_label_generator_user.tx</code>
Script	<code>scripts/label_bertopic_clusters.py</code>
Temperature / seed	<code>0.0 / 42</code>

Label generator — system (verbatim):

You are building an occupational task taxonomy. Your job is to read a set of AI tool descriptions that have already been confirmed to represent occupational work, and write a task description in the style of O*NET -a US occupational database that describes what workers do on the job.

O*NET task descriptions:

- Begin with an active verb (e.g., Analyze, Develop, Prepare, Monitor, Coordinate)
- Describe what a worker does, not what a software tool does
- Are -1020 words long -concise, like actual O*NET tasks
- Use specific, domain-relevant vocabulary -avoid generic language
- Capture the purpose or outcome of the task, not just the action
- Do NOT mention specific software, tool names, or product names
- Always write in English, regardless of the language of the input descriptions

Respond in this exact format:

Label: [short label, -36 words]

Description: [O*NET-style description, -1020 words]

Label generator — user template (verbatim):

Here are {n} tool descriptions from a cluster of related AI tools, along with the top keywords that distinguish this cluster. This cluster has already been confirmed to represent occupational work.

Top keywords: {keywords}

Tool descriptions:
{tool_descriptions}

Write:

1. Short label -(36 words): A concise name for the task category this cluster represents.
2. O*NET-style description -(1020 words): What does a worker do when performing this type of task? Start with an active verb. Be concise -match the style and length of real O*NET task descriptions.

Respond in this exact format:

Label: [your label]

Description: [your description]

Example 1:

Top keywords: sql, executequery, queries, execute, select, executesql, executes, query, soql, runquery

Tool descriptions:

1. Execute a SQL query and return the results in a formatted table.
2. Execute with typed input (sql, parameters, row_limit, format: markdown or json)
3. Execute AQL queries against a database (supports offset and fetch)
4. Execute a saved Look; supports json, csv, xlsx, and sql output
5. Execute a CQL query on a specific node

Label: Database query execution

Description: Execute SQL commands against relational databases to retrieve, filter, and export records for analysis.

Example 2:

Top keywords: protein, gene, genes, genome, proteins, sequences, sequence, uniprot, pdb, molecular

Tool descriptions:

1. Pathway enrichment analysis on a list of identifiers
2. Generate reverse complement of DNA sequences
3. Experimental protein structures
4. Find expression datasets (RNA-seq, microarray)
5. Get disease associations for a protein

Label: Genomic and proteomic data analysis

Description: Analyze genomic sequences and proteomic datasets to identify gene expression patterns and molecular disease associations.

Example 3:

Top keywords: healthcheck, health, connectivity, check, gethealth, checkhealth, healthy, system, status, verify

Tool descriptions:

1. Comprehensive server health check with metrics aggregation
2. Shows subsystem health, API key configuration, cache location, and sync status
3. Check if the mirror is reachable and measure latency
4. Fast health check
5. Get system health status

Label: System health monitoring

Description: Monitor server and application health by querying status endpoints and measuring latency to detect service degradation.

Example 4 (professional, not personal -note the organizational context):

Top keywords: readnote, getnote, note, getnotecontent, getnoteinfo, notes, notesget, frontmatter, read, enml

Tool descriptions:

1. Fetch full content of a transcript concept note
2. Get full note data (fields, tags, cards)

3. Returns note content identified by its ID
4. Read the complete content of a note by its relative path
5. Get note by UUID or title

Note: These tools operate in an organizational knowledge management system -fetching structured records by ID, path, or UUID -not personal journaling.

Label: Knowledge base note retrieval

Description: Retrieve stored notes and knowledge base entries by identifier, title, or file path to support information review.

Now complete the task for the cluster above.

A.6 Gate (occupational classification)

Same v6 labeling run as §A.5. Categories: OCCUPATIONAL / PLUMBING / CONSUMER / MICRO / INCOHERENT (1,139 / 336 / 188 / 36 / 13). Shared criteria file plus gate generator; evaluator prompts: prompts/step2_v5_gate_evaluator_*.txt; gate adjudicator: prompts/step2_v5_adjudicator_*.txt.

Model	command-a-plus-05-2026 (Model Vault)
Criteria	prompts/step2_v6_criteria.txt
Gate generator system	prompts/step2_v5_gate_generator_system.txt
Gate generator user	prompts/step2_v6_gate_generator_user.txt
Manifest	outputs/bertopic_results/cluster_labels_v6.manifest.json

Shared criteria (verbatim):

A cluster is OCCUPATIONAL only if the tool descriptions show work that a professional would perform as part of a job, in a professional or organizational context. Otherwise it belongs to exactly one of four exclusion categories:

****PLUMBING**** -internal software mechanics, not worker tasks: cache invalidation, webhook registration, retry/backoff logic, memory and session cleanup, process termination, random number generation, string manipulation, CAPTCHA solving, isolated connectivity utilities (checking whether a port is open, pinging a host, echoing input). Test: would any job posting list this as a duty? IMPORTANT distinction: a cluster is an offensive security toolkit (OCCUPATIONAL -penetration testing, 0*NET 15-1299.04) only when it contains MULTIPLE DISTINCT offensive-security capabilities used together -e.g., vulnerability scanning plus password auditing plus web application probing (tools like masscan, hydra, wpscan, hashcat appearing as a suite). A cluster of single-purpose connectivity utilities -checking whether a port is open, pinging hosts, testing reachability -is PLUMBING, even though port checks can be one step in security work. The test is the cluster's breadth of offensive capability, not the presence of the word "scan."

****CONSUMER**** -personal or lifestyle activities done for oneself, not as professional service: personal journaling and note-taking, personal reminders and to-do lists, stress-relief breaks, meal/sleep/habit tracking for oneself, horoscope and fortune-telling, torrent downloading, personal playlist curation, liking posts or reacting with emoji personally. IMPORTANT: professions adjacent to these activities exist (astrologers, dietitians, social media managers), but the category is decided by the TOOLS' context: if the tool descriptions show personal consumption rather than professional service delivery in an organizational context, it is CONSUMER. A meal-logging app is CONSUMER even though dietitians exist; a brand's social media scheduling platform is OCCUPATIONAL.

****MICRO**** -single UI gestures too atomic to be an occupational task: clicking a button, scrolling, dragging an element, marking one item read, toggling a reaction.

****INCOHERENT**** -tool descriptions too short, inconsistent, or unclear to characterize; or predominantly non-English with unclear content (do not guess from foreign-language keywords). A computed language statistic is provided. If a majority of the cluster's descriptions are non-English AND the sampled descriptions do not share one clear task theme, the cluster is language-clustered rather than task-clustered: classify INCOHERENT. Do not infer a task theme from topically unrelated tools that merely share a language.

Judge ONLY from the tool descriptions and keywords. Ignore any professional-sounding framing in a proposed label: plumbing described in enterprise language is still plumbing.

Gate generator — system (verbatim):

You are building an occupational task taxonomy from clusters of AI tool descriptions. Your job is to decide whether a cluster represents occupational work or should be excluded.

{criteria}

Respond in this exact format:

Category: [OCCUPATIONAL, PLUMBING, CONSUMER, MICRO, or INCOHERENT]

Reasoning: [one or two sentences citing specific tool descriptions]

Gate generator — user template (verbatim):

Top keywords: {keywords}

Language statistics (computed over all {n_total} tool descriptions in this cluster): {pct_non_english}% non-English (top languages: {top_languages}).

Tool descriptions ({n} sampled from cluster of {size} tools):

{tool_descriptions}

Which category does this cluster belong to?

A.7 Three-way work-type judgment

Production: outputs/new_vs_subatomic_v4.csv (SUBATOMIC 693 / COMPOSITE 411 / NEW 35). Judged object per cluster: **cluster description + label + top-5 nearest software-performable O*NET statements** (v5 universe, 9,423), not member tool names (outputs/repo_rt_cleanup_lookups.md).

Model	command-a-plus-05-2026 (Model Vault chat); embed embed-multilingual-v3.0 (staging)
Prompts	prompts/step3a_v4_generator_system.txt, step3a_v4_generator_user.txt
Manifest	outputs/new_vs_subatomic_v4.manifest.json
Script	scripts/judge_new_vs_subatomic_v4.py
Shortlist universe	data/processed/onet_tasks_computer_v5.jsonl
Temperature / seed	0.0 / 42

Generator — system (verbatim):

You classify AI tool clusters against the occupational record. You see a cluster's description and label, plus the most semantically similar software-performable O*NET tasks (from our 9,423-task universe). Choose one of three judgments.

****SUBATOMIC**** -the cluster is a component or sub-step of an existing task, at a finer grain than O*NET breaks out. A single plausible parent exists. This is by far the most common case.

- "Deletes media files and associated metadata from libraries, posts, and archives." -one operation applied across several targets.
- "Locate users by name, email, or username to support communication and data retrieval." -one lookup, described with its input fields and purposes.
- "Analyze call stack frames to identify program execution flow and locate errors" -one analysis with its purpose attached; a step inside debugging, not debugging itself.

Note all three list multiple things and are still SUBATOMIC. Listing targets, input options, or the purpose a step serves does not make a cluster composite. Only multiple distinct *activities* do.

****COMPOSITE**** -recognizable occupational work, but not a sub-step of any single task. It spans several tasks or bundles them into a workflow O*NET does not name.

- "Track student enrollment, assignment submissions, and GPA trends to inform academic advising decisions." - tracking and advising are separate activities with different outputs.
- "Orchestrate and execute end-to-end data pipelines, managing validation, transformation, and audit processes ." -distinct stages run in sequence, not one of them performed.

The test: reduce the description to its verbs and ask whether they are one operation or several. If a single parent task covers the whole cluster, choose SUBATOMIC even when the description is long.

****NEW**** -the work has no plausible counterpart in the occupational record, because it did not exist before AI agents did. That absence is the whole test.

Two kinds have recurred in earlier manual labelling. They are orientation, not a closed list:

1. ***Building automated systems*** -constructing automation, usually by developers. The output is a mechanism that completes tasks, not a completed task.
 - "Automate web page navigation, data extraction, and interaction to gather information and capture screenshots."
 - "Develop, configure, and deploy applications and serverless functions, managing environments, domains, and automated tasks."
2. ***Working with and managing agents*** -work that appears once agents are running, in any occupation, not only software ones. The object is an agent, model or automated process: creating and switching between them, configuring behaviour, monitoring, coordinating, or judging whether one can be trusted.
 - "Develop and manage AI personas, including creation, activation, and switching for assistant interactions ."
 - "Configure and interact with language models to generate responses, manage parameters, and monitor performance."
 - "Select and configure voice options for text-to-speech synthesis, adjusting language, gender, and speaker characteristics."

****If a cluster has no occupational counterpart but fits neither kind, still judge NEW.**** Record the kind as OTHER and name the work in a short phrase. Do not force it into a kind, and do not retreat to SUBATOMIC or COMPOSITE because it matches neither.

Resembling a kind is not sufficient. Familiar work that merely involves software is SUBATOMIC or COMPOSITE.

NEW takes precedence over COMPOSITE. Several NEW examples above list multiple activities and would read as composite on the verb test; apply the no-counterpart test first. The verb test decides only between SUBATOMIC and COMPOSITE.

Two failure modes

****A new technical substrate is not new work.**** Containers, blockchains, browsers, model APIs -managing configuration in a container platform is still managing configuration. Ask what the work accomplishes, not what it runs on. Dense jargon makes familiar work look unprecedented, so translate into plain terms first.

****A poor shortlist is not evidence of novelty.**** Retrieval is by text similarity, so it sometimes returns an unrelated occupation that shares vocabulary -container cleanup retrieving Stockers and Order Fillers on "removing" and "stock". That is a retrieval failure and says nothing about whether a parent exists elsewhere in O*NET. Judge against your own knowledge of occupational work instead, and record it in "Shortlist usable".

Be conservative about NEW: prefer SUBATOMIC when a single parent fits, COMPOSITE when the work is recognizable but no single parent does.

Respond in this exact format:

Judgment: SUBATOMIC or COMPOSITE or NEW
 Parent task: [exact O*NET task text if SUBATOMIC; the closest one or two if COMPOSITE; else NONE]
 New kind: [BUILDING_AUTOMATION, WORKING_WITH_AGENTS, or OTHER if NEW, else NONE]
 Other kind description: [short phrase if New kind is OTHER, else NONE]
 Shortlist usable: [YES or NO]
 Reasoning: [one sentence]

Generator — user template (verbatim):

****Cluster description:**** {onet_description}

```
**Cluster label:** {label}
```

```
**Top 5 nearest software-performable O*NET tasks:**  
{shortlist}
```

Is this a sub-step of one of these tasks (SUBATOMIC), recognizable work spanning or sitting above them (COMPOSITE), or work with no plausible occupational counterpart because it exists only now that AI agents do (NEW)? If NEW, name the kind. If the shortlist is from the wrong occupational territory, treat it as unreliable rather than as evidence of novelty.

A.8 Agent-dependence judgment

Production: data/processed/clusters_agent_dependence_v1.csv via scripts/step6b_novelty_and_agency_v1.py.

Model	command-a-plus-05-2026 (Model Vault)
Prompts	prompts/agent_dependence_v1_system.txt, agent_dependence_v1_user.txt
Manifest	outputs/step6b_novelty_and_agency_v1.manifest.json
Output	data/processed/clusters_agent_dependence_v1.csv

System (verbatim):

You judge whether a described work task would exist independently of AI agents.

Would this work exist as a recognisable job task if AI agents did not exist?

Answer PRE_EXISTING if people were doing this work, in some form, before autonomous AI agents; answer

AGENT_DEPENDENT if the work only exists because autonomous agents exist and there is no human counterpart to it.

Respond as:

Verdict: PRE_EXISTING or AGENT_DEPENDENT

Reasoning: [one sentence citing the specific feature of the description that decided it]

User template (verbatim):

Work task description:

{task_text}

Judge whether this work is PRE_EXISTING or AGENT_DEPENDENT. Respond in the exact format specified.

A.9 Cluster-to-occupation crosswalk

Production: outputs/bertopic_results/cluster_to_occupation_v2.csv. Generator proposes an O*NET occupation or NEW_OCCUPATION; evaluator is an LLM rubric (Gen/Eval loop, ≤ 3 iterations) recorded in the evaluator prompts—not a separate non-LLM script. Embeddings for retrieval: embed-multilingual-v3.0.

Chat model	command-a-plus-05-2026 (Model Vault)
Embed model	embed-multilingual-v3.0 (staging)
Generator	prompts/step3b_v2_generator_system.txt, step3b_v2_generator_user.txt
Evaluator	prompts/step3b_v2_evaluator_system.txt, step3b_v2_evaluator_user.txt
Manifest	outputs/bertopic_results/cluster_to_occupation_v2.manifest.json
Script	scripts/crosswalk_clusters_to_occupations.py
Seed / T / max loops	42 / 0.0 / 3

Generator — system (verbatim):

You are assigning AI tool clusters to occupational categories. You will see a cluster's task description and a list of candidate O*NET occupations retrieved by semantic similarity. Your job is to select the best-fitting occupation, or declare that none fits.

Rules:

- Select the occupation whose workers would most plausibly perform the work described by the cluster.
- If the cluster is marked SUBATOMIC, the first candidate is the occupation of the parent O*NET task -treat it as a strong prior, but override it if a different occupation is a clearly better fit.
- If none of the candidates is a convincing fit -the cluster describes work that does not belong to any existing occupation -respond with NEW_OCCUPATION. This is a valid and expected output, particularly for clusters marked NEW.
- Do not force a match. A confident NEW_OCCUPATION is better than a weak occupation assignment.

Respond in this exact format:

SOC: [the SOC code from the candidate list, e.g. 15-1252.00, or NEW_OCCUPATION]

Occupation: [occupation title, or NEW_OCCUPATION]

Reasoning: [one sentence]

If (and only if) you responded NEW_OCCUPATION, add two lines:

Proposed title: [a -36 word occupation title, in the style of O*NET occupation titles, for the occupational category this work would belong to]

Proposed rationale: [one sentence: what distinct occupation this work implies]

Decide SOC-vs-NEW_OCCUPATION exactly as above first. The proposed title is generated only after that decision and must not be used to justify it.

Generator — user template (verbatim):

```
**Cluster label:** {label}
**Cluster description:** {onet_description}
**Classification:** {judgment} {classification_note}

**Candidate occupations (ranked by semantic similarity):**
{candidates}

Which occupation best fits this cluster, or is this NEW_OCCUPATION?
```

Evaluator — system (verbatim):

You are reviewing an occupation assignment for an AI tool cluster. Your job is to decide whether the proposed assignment is convincing, or whether it should be rejected.

Accept the assignment if: the proposed occupation's workers would plausibly perform the work described by the cluster, and the reasoning is coherent.

Reject the assignment if: the occupation is a poor fit, the reasoning is circular or vague, or a clearly better candidate exists among the options that was overlooked. If the assignment is NEW_OCCUPATION, reject it if any of the candidate occupations is actually a convincing fit.

Respond in this exact format:

Decision: ACCEPT or REJECT

Feedback: [one sentence -if ACCEPT, briefly confirm why; if REJECT, give specific feedback the Generator can use to revise]

Evaluator — user template (verbatim):

```
**Cluster label:** {label}
**Cluster description:** {onet_description}
**Classification:** {judgment}

**Candidate occupations that were available:**
{candidates}

**Proposed assignment:**
SOC: {soc_code}
Occupation: {occupation_title}
Reasoning: {reasoning}
```

Is this assignment convincing?

A.10 EPOCH-style capability ratings

Production: `data/processed/clusters_epoch_style_v1.csv`. Each occupational cluster description is scored on E/P/O/C/H on a 0–4 scale, then averaged across six text versions (original + five paraphrases from `prompts/sfi_paraphrase_v1_*.txt`; `epoch_style_n_versions=6` for 1,138/1,139 clusters in the v5 taxonomy package).

Model	<code>command-a-plus-05-2026</code> (Model Vault)
Scoring prompts	<code>prompts/epoch_style_scoring_v1_system.txt</code> , <code>epoch_style_scoring_v1_user.txt</code>
Paraphrase prompts	<code>prompts/sfi_paraphrase_v1_system.txt</code> , <code>sfi_paraphrase_v1_user.txt</code>
Averaging	mean over 6 versions (original + 5 paraphrases)
Scale	0=not involved ... 4=central (per dimension)
Manifest	<code>outputs/step6b_novelty_and_agency_v1.manifest.json</code>
Script	<code>scripts/step6b_novelty_and_agency_v1.py</code>

Scoring — system (verbatim; includes 0–4 definitions):

You rate work task descriptions on five human capability dimensions. These are capabilities that complement rather than substitute for AI. Rate each dimension 0 to 4 for how central that capability is to performing the described work: 0 = not involved at all, 1 = marginally involved, 2 = somewhat involved, 3 = substantially involved, 4 = central to the work.

****E -Empathy and Emotional Intelligence.**** Fostering understanding, teamwork, and a supportive collaborative environment. Includes connective labour: building relationships and creating conditions for shared goals. Requires mutual, reciprocal interaction, and empathy that is genuine and perceived as genuine. Occupations high on this: social workers, healthcare workers, teachers. Score 0 if the work involves no other person's emotional state.

****P -Presence, Human Connection and Networking.**** Capabilities used when physically present in a workspace: face-to-face interaction, spontaneous collaboration, and physical capabilities such as dexterity, touch, and lived experience of a place. Occupations high on this: nurses, hairdressers, on-the-ground journalists. Score 0 if the work can be performed entirely remotely and involves no physical manipulation.

****O -Opinion, Judgment and Ethics.**** Critical thinking, synthesising information, weighing diverse perspectives, and combining rational analysis with intuition and moral consideration to decide or resolve. Includes accountability, responsibility, agency, and self-determination, and the ability to act in open-ended situations where the right answer is not determined by rules. Occupations high on this: judges, scientists. Score 0 if the work follows determinate rules with no discretion.

****C -Creativity and Imagination.**** Producing novel and original ideas, and envisioning possibilities beyond what exists. Includes curiosity, the willingness to challenge existing framings, improvisation, humour, and design reasoning -forming hypotheses about how things might be rather than selecting from a known set. Score 0 if the work produces a determinate output from a determinate input.

****H -Hope, Vision and Leadership.**** Optimism, initiative, grit and perseverance; forming a goal and inspiring others toward it; supplying purpose and direction. Score 0 if the work involves no goal-setting for others and no sustained pursuit of an outcome against difficulty.

Rate the work as described, not the software that might perform it. Respond in this exact format, with no commentary:

E: [0–4]

P: [0–4]

O: [0–4]

C: [0–4]

H: [0–4]

Reasoning: [one sentence naming the dimension that scored highest and why]

Scoring — user template (verbatim):

Work task description:

{task_text}

Rate this work on E, P, O, C, and H -(04 each) using the system rubric. Respond in the exact format specified.